The Optimizers Everyone Uses, Finally Proven to Converge

Ruinan Jin, Yu Xing, and Xingkang He establish almost-sure convergence of momentum SGD and standard AdaGrad on non-convex losses (arXiv 2201.11204)

Almost every deep model you have ever trained was optimized by a method whose behavior nobody could fully prove. Plain stochastic gradient descent is well understood, but the two variants that practitioners actually reach for, momentum SGD (mSGD) and AdaGrad, sit on shakier theoretical ground. Momentum smooths the update direction by mixing in a running average of past gradients; AdaGrad rescales each step by the accumulated size of the gradients it has seen. Both reliably beat vanilla SGD in practice. Both were, until recently, only partially justified in theory.

The gap is not academic hairsplitting. For the smooth, possibly non-convex loss functions that describe real neural networks, the strongest existing results for these methods guarantee only weak forms of convergence: that some subsequence of the iterates settles down, or that a time-average of the iterates does. Neither statement promises that the trajectory you actually run will approach a stationary point. That stronger, per-run guarantee, holding with probability one, is exactly what a practitioner implicitly assumes every time they hit train.

This paper closes much of that gap. Working with the static momentum coefficient people really use and the standard norm-form AdaGrad people really deploy, the authors prove that both methods converge almost surely to a connected set of stationary points, even when the loss is non-convex. They also quantify how fast mSGD gets there, turning the folklore that momentum accelerates SGD into a statement you can read off a formula.

Paper: https://arxiv.org/abs/2201.11204

The gap between what we run and what we can prove

Two mismatches separate the textbook analyses from deployed practice. First, most convergence proofs for momentum methods require the momentum coefficient to shrink toward zero over time, whereas practitioners fix it at a static value such as 0.9 and never touch it again. Second, the AdaGrad results in the literature often cover a modified version of the algorithm rather than the standard norm-based update that ships in optimizers. On top of both, classical proofs lean on assumptions that non-convex learning violates: convexity of the loss, iterates that stay in a bounded region, or gradient noise whose magnitude is uniformly bounded over the entire parameter space.

That last assumption is more fragile than it looks. A uniform bound on the gradient noise simply fails for objectives as ordinary as a quadratic or a cubic, whose gradients grow without limit as the parameters move away from the origin. Any theory that needs it has quietly excluded much of the loss landscape it claims to describe. The class of losses this paper targets is

deliberately permissive: non-negative, continuously differentiable functions with Lipschitz gradients, allowed to be non-convex and to carry many separate critical points.

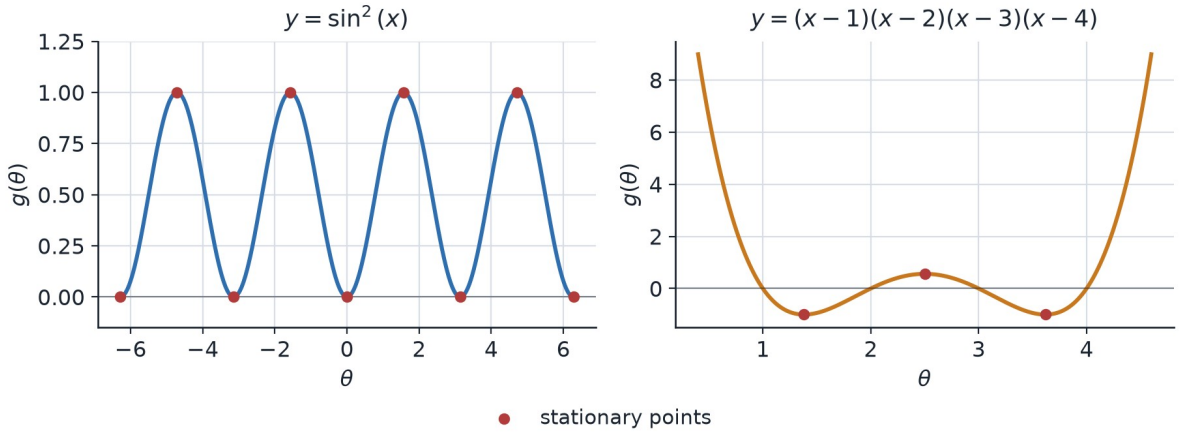

Figure 1. Two non-convex objectives the paper names as examples covered by its assumptions: the periodic sin²(x) and the quartic (x-1)(x-2)(x-3)(x-4). Both have several stationary points (red), the multi-basin structure the convergence theorems must handle.

The tool: treating the optimizer as a noisy dynamical system

The analysis rests on stochastic approximation, the classical idea that a noisy iterative update can be read as a discrete-time approximation of an underlying continuous dynamical system. Seen this way, mSGD and AdaGrad are not just update rules but trajectories that should asymptotically track a gradient flow toward the set of stationary points. The job of the proof is to show that the noise does not knock the trajectory off that flow in the long run.

Two ingredients make that possible. A stability lemma keeps the expected loss bounded throughout training, so the iterates cannot escape to infinity. And a relaxed noise condition replaces the brittle uniform bound with a milder one: the gradient variance is allowed to grow, but only in proportion to the loss value itself, of the form $E\|\nabla g(\theta) - \nabla g(\theta,\xi)\|^2 \le M(1 + g(\theta))$. Where the loss is large, more noise is tolerated; where the loss is small and precision matters, the noise is reined in. This is exactly the behavior a quadratic or cubic objective needs and a uniform bound cannot provide.

For mSGD, the randomness is tamed by a decreasing step size that satisfies the classical Robbins-Monro conditions, its sum diverging while the sum of its squares stays finite, with the canonical choice $\varepsilon_n = 1/n$. Crucially the momentum coefficient stays static while the step size does the work, matching how the method is used rather than how it is usually analyzed.

Result 1: momentum SGD converges almost surely

The first main theorem states that the iterates of mSGD converge, with probability one, to a connected set of stationary points of the loss. When that set collapses to a single point, the iterates converge to that point outright. This is a genuinely stronger claim than the subsequence-convergence results it supersedes: it is not that some cleverly chosen

subsequence behaves well, but that the whole trajectory does, for essentially every realization of the training noise.

Why momentum accelerates, made precise

Beyond convergence, the paper quantifies the rate. The expected squared gradient norm of mSGD decays on the order of an exponential whose exponent scales with the accumulated step size divided by $(1-\alpha)^2$. The momentum coefficient enters entirely through that $(1-\alpha)^2$ term, which gives a clean way to see its effect: the multiplier $1/(1-\alpha)^2$ sits in front of the exponent, so a larger α makes the exponent bigger and the bound decay faster.

Two limits pin the picture down. Set $\alpha = 0$ and the multiplier is 1: the rate reduces exactly to the known rate of plain SGD, a reassuring consistency check. Push α toward 1 and the multiplier grows without bound, and the corresponding time-average convergence rate improves to order O(1/T). This is the analytic counterpart of an ablation study. Where an empirical paper would sweep α and plot accuracy, this one reads the dependence straight off the rate.

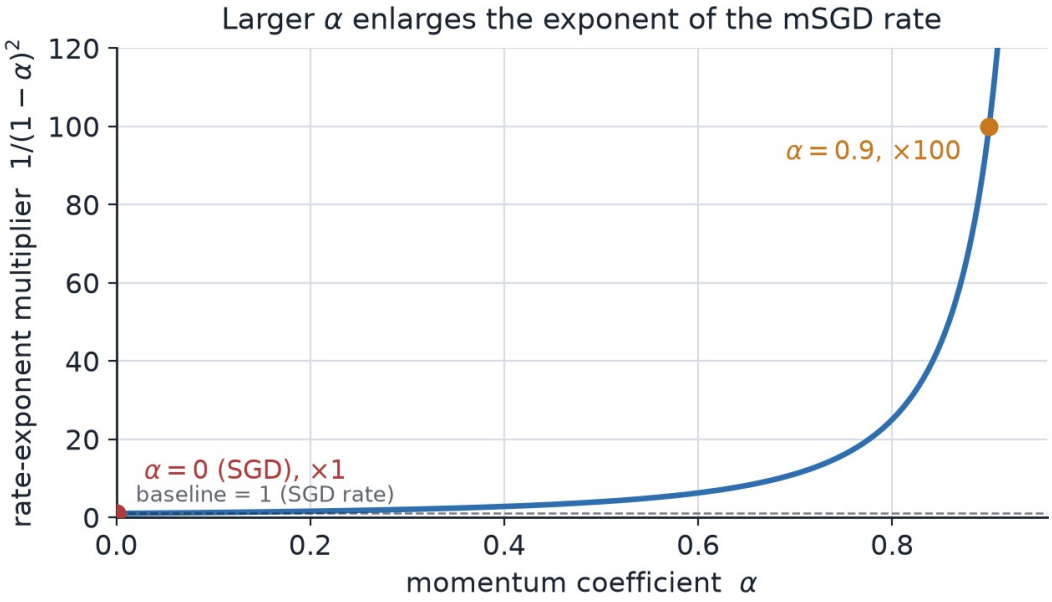

Figure 2. The momentum-dependent multiplier $1/(1-\alpha)^2$ that scales the exponent of the mSGD rate. At $\alpha = 0$ it equals 1 and the bound matches SGD; at $\alpha = 0.9$ it is 100, and it diverges as α approaches 1, the regime where the time-average rate reaches O(1/T).

Result 2: standard AdaGrad converges too

The second main theorem extends the same almost-sure asymptotic convergence to standard norm-form AdaGrad, again for non-convex losses. Reaching it takes more work than mSGD for a specific reason: AdaGrad has no externally imposed decreasing step size to counteract the noise, and its adaptive step size is itself a random quantity that depends on the current gradient noise. That dependence breaks the naive conditional-expectation manipulations the momentum analysis relies on, because the step and the gradient it multiplies are no longer independent given the past. The authors develop bounds that control exactly

these conditionally dependent terms, lifting AdaGrad's guarantee from subsequence convergence up to convergence of the full trajectory.

What the proof gives up, and what it keeps

The table below summarizes how the paper's conditions compare with the standard toolkit. The pattern is consistent: assumptions that fail for realistic non-convex training are dropped, and what remains are conditions that a practitioner can actually verify in real settings.

Table 1. Assumptions relaxed relative to common prior analyses.

Assumption	Common prior analyses	This paper
Loss convexity	Often required	Not required (non-convex allowed)
Bounded iterates	Often assumed	Not assumed
Gradient noise	Uniformly bounded over space	Bounded relative to loss: M(1 + g(θ))
Momentum coefficient	Time-varying, shrinking to 0	Static (e.g. 0.9)
AdaGrad form	Modified variants	Standard norm form
Convergence mode	Subsequence / time-average	Almost-sure asymptotic

Takeaway

The message is a reassuring one for anyone who trains models. Under mild and realistic conditions, both momentum SGD with a fixed momentum coefficient and standard AdaGrad are guaranteed to converge, almost surely, to stationary points of general non-convex losses, and momentum provably accelerates SGD rather than merely appearing to. By shedding the convexity, bounded-iterate, and uniform-noise assumptions that earlier proofs carried, the work narrows a long-standing gap between the strong empirical track record of these optimizers and the theory meant to explain them.

The honest boundaries are worth stating. This is a purely theoretical contribution, with no experiments and no benchmark numbers, and its guarantees are asymptotic: they describe where the iterates end up, not how few steps it takes to get close, and convergence to a stationary set is not the same as convergence to a global minimum. Within those limits, it does something valuable, giving two of the field's most-used optimizers the kind of convergence footing that plain SGD has long enjoyed.